

Section 1

2026年1月23日 星期五

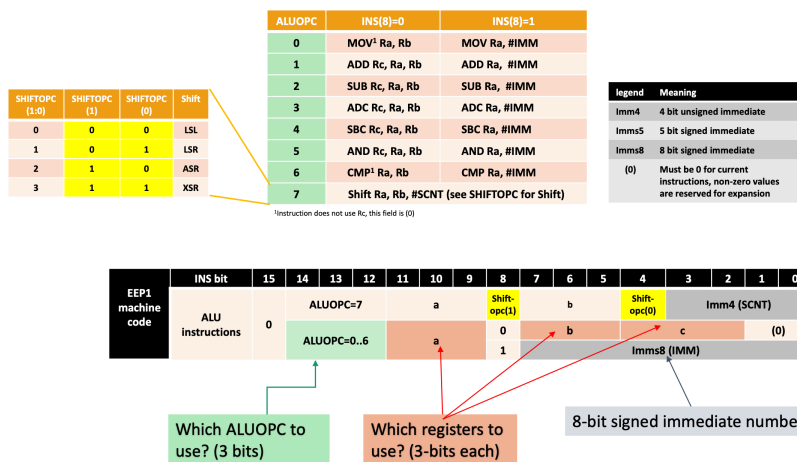
15:20

1 Register-register MOV instructions & the register file

Tasks

Task 1. Using the [environment setup guide](#) for support, convert the assembly code in Figure 1 into machine code and load it into eep1lab1. For all of Lab 1 you will use eep1lab1 as your EEP1 design.

How to read EEP1 Assembly Language



18

We have machine code like this.

```
MOV R6, #3
MOV R7, R6
```

Figure 1: Assembly language program of two EEP1 MOV instructions

Run the simulation and use the waveform viewer (see the environment setup guide) to check that register values change as expected. Use codemem.dout to see the instruction word being executed each cycle.

So MOV R6, #3 is 0 000 110 1 00000011

Which is 0D03

MOV R7, R6 is 0 000 111 0 11000000

Which is 0EC0

	0	1
Eep1.CODEMEM.Dout(15:0)	x0D03	x0EC0

For these MOV instructions, do the registers change in the same cycle as the instruction that changes their values, or in the following cycle?

	0	1	2	3	0
Eep1.CODEMEM.Dout(15:0)	x0D03	x0D03	x0EC0	x0000	x0D03
Controlpath.PC.Q(15:0)	x0000	x0000	x0001	x0002	x0003
Reg16x8.REG2.Q(15:0)	x0000				x0000
Reg16x8.REG0.Q(15:0)	x0000				x0000
Reg16x8.REG1.Q(15:0)	x0000				x0000
Reg16x8.REG3.Q(15:0)	x0000				x0000
Reg16x8.REG4.Q(15:0)	x0000				x0000
Reg16x8.REG5.Q(15:0)	x0000				x0000
Reg16x8.REG6.Q(15:0)	x0000	x0003			x0003
Reg16x8.REG7.Q(15:0)	x0000	x0000	x0003		x0003

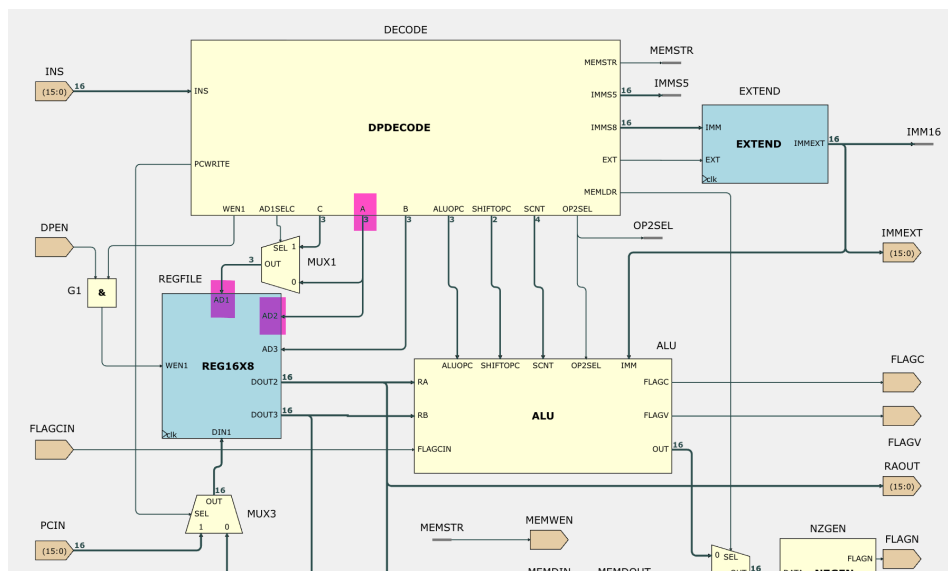
In the following cycle as R6 changes in cycle1 & R7 changes in cycle2.

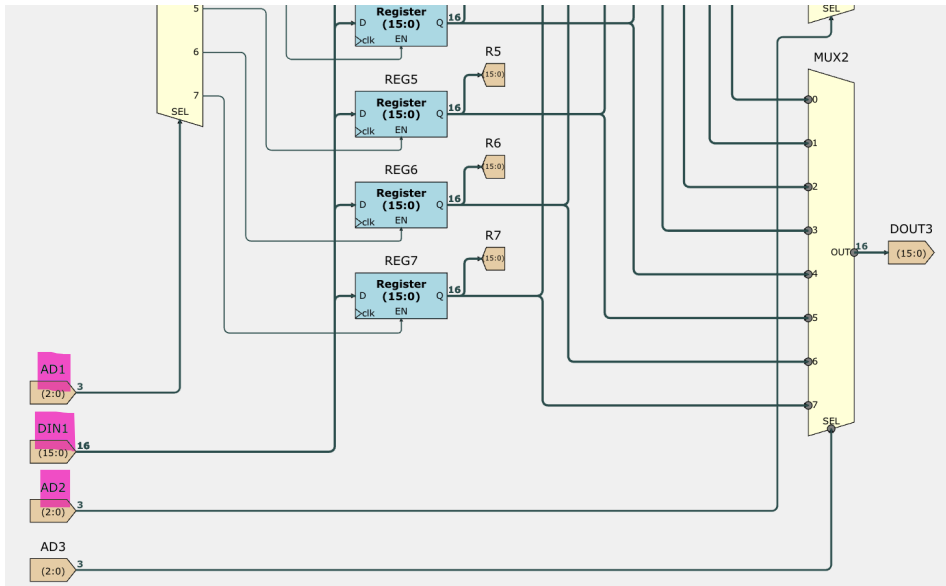
Using the EEP1 documentation ([ALU Instructions : detail](#)), confirm that for register-register MOV instructions (where INS(8) = 1) the field a (written as Ra in assembly) determines which register is written in the register file. Then, using the ALU Instruction Encoding sheet, state exactly which bits of the 16-bit instruction word form field a.

Field a determines 9-11 registers. For Ra = R6, it is 110.

	0	1	0
Controlpath.PC.Q(15:0)	b0000,0000,0000,0000	b0000,0000,0000,0001	b0000,0000,0000,0000
Eep1.CODEMEM.Dout(15:0)	b0000,1101,0000,0011	b0000,1110,1100,0000	b0000,1101,0000,0011
Reg16x8.REG2.Q(15:0)	b0000,0000,0000,0000		b0000,0000,0000,0000
Reg16x8.REG0.Q(15:0)	b0000,0000,0000,0000		b0000,0000,0000,0000
Reg16x8.REG1.Q(15:0)	b0000,0000,0000,0000		b0000,0000,0000,0000
Reg16x8.REG3.Q(15:0)	b0000,0000,0000,0000		b0000,0000,0000,0000
Reg16x8.REG4.Q(15:0)	b0000,0000,0000,0000		b0000,0000,0000,0000
Reg16x8.REG5.Q(15:0)	b0000,0000,0000,0000		b0000,0000,0000,0000
Reg16x8.REG6.Q(15:0)	b0000,0000,0000,0000	b0000,0000,0000,0011	b0000,0000,0000,0000
Reg16x8.REG7.Q(15:0)	b0000,0000,0000,0000		b0000,0000,0000,0000

Open the regfile sheet. Examine the names and widths of its input and output ports. From the port names (and the hardware they drive), deduce which input port controls the address of the register that is written. You should find that this port is AD1.





Explain how this hardware is arranged so that the value on AD1 selects which register the incoming data is written to.

AD1 connects to a DEMUX in Regfile to control which address from R0 to R7 to write in.
And the data that writes in is in DIN1, so every time EN rises, $RA \leftarrow DIN1$.
Finally, AD2 connects to select of DOUT2 and outputs value in R6 to DOUT2

vi. Using the waveform viewer, track the value of AD1 in each cycle.
Does it correspond to the register that is written in the next cycle, as you expect?

	0	1	2	0
Datapath.REGFILE.Ad1(2:0)	6	7	0	6
Eep1.CODEMEM.Dout(15:0)	x0D03	x0EC0	x0000	x0D03
Reg16x8.REG2.Q(15:0)	x0000		x0000	x0000
Reg16x8.REG0.Q(15:0)	x0000		x0000	x0000
Reg16x8.REG1.Q(15:0)	x0000		x0000	x0000
Reg16x8.REG3.Q(15:0)	x0000		x0000	x0000
Reg16x8.REG4.Q(15:0)	x0000		x0000	x0000
Reg16x8.REG5.Q(15:0)	x0000		x0000	x0000
Reg16x8.REG6.Q(15:0)	x0000	x0003		x0003
Reg16x8.REG7.Q(15:0)	x0000	x0003	x0003	x0003

Ad1 is 6; 7, which is as expected

2. Read address control. Identify the hardware that controls which register the data is read from in a register-to-register MOV instruction such as in Figure 1. Again, explain how it works in detail.

Use a similar approach to Question 1. A suggested (shorter) method is:

i. Using [ALU Instructions : detail](#), confirm that for a MOV instruction with $INS(8) = 0$ the field b (written as Rb) selects the register from which data is read in the register file. Using the ALU Instruction Encoding sheet, identify exactly which bits of the instruction form field b.

EEP1 machine code	INS bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
	ALU instructions	0	ALUOPC=7				a		Shift- opc(1)		b		Shift- opc(0)		Imm4 (SCNT)			
			ALUOPC=0..6				a		0		b		c		(0)			
									1				Imms8 (IMM)					

Again from encoding sheet, b is read from 5-7 bits of instruction code.

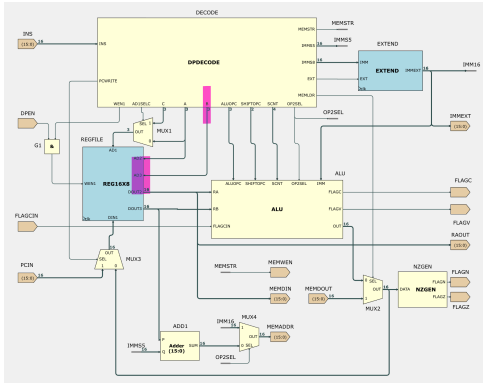
In the waveform viewer inspect the second instruction word (the one in cycle x01) in binary. Read the

In the datapath viewer, inspect the second instruction word (the one in cycle 1), in binary. Read the read address from this field and check that it matches the source register in the assembly instruction.

	0	1	2	3	0
Datapath.DECODE.B(2:0)	b000	b110	b000	b000	b000

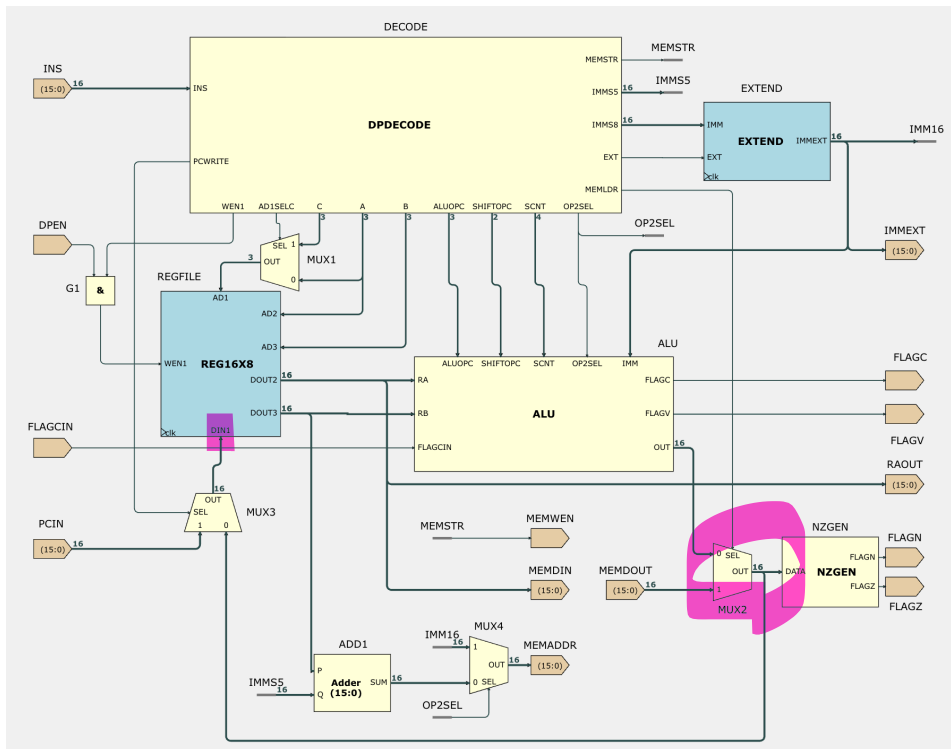
Rb outputs as B in DPDECODE, and it is b110 (6) in cycle 1, which is correct for MOV R7, R6.

Open dpdecode and trace the wire that carries the bits of field b. Note which port of the register file it connects to. This port is the one that controls the read address for these register-to-register MOV instructions.

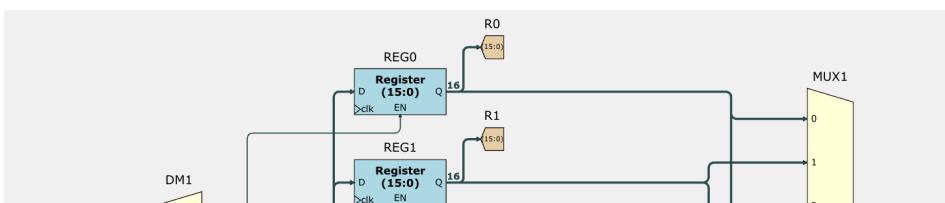


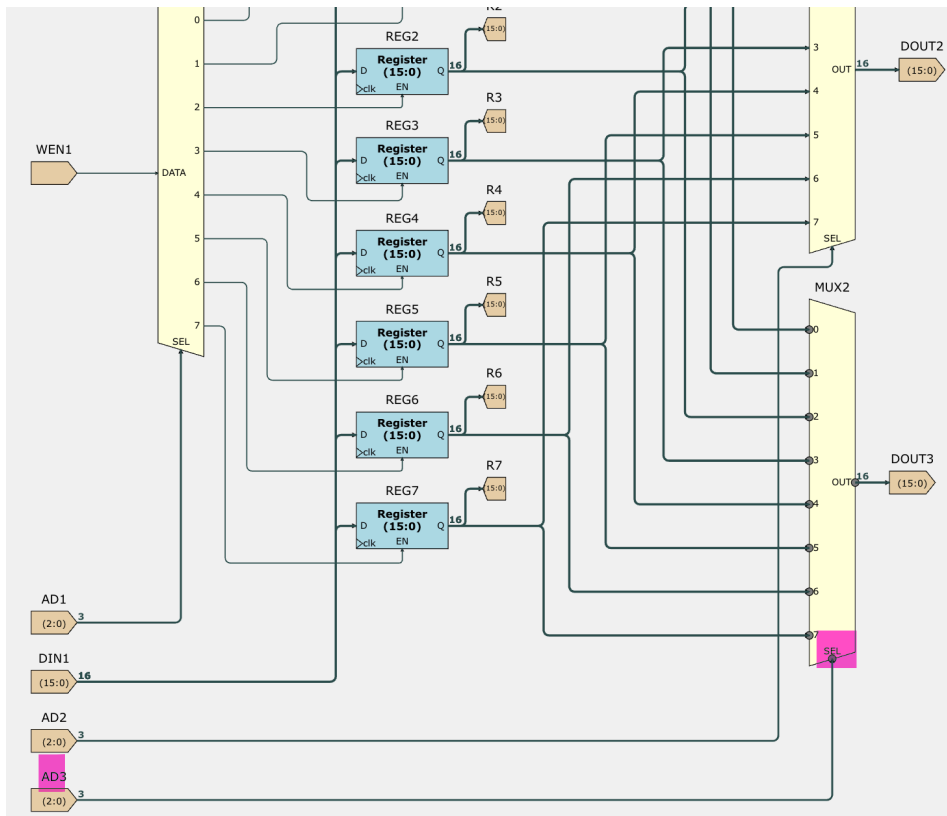
It comes out of B and goes to AD3

iii. Open the register file and follow the hardware connected to that read-address port. Explain how the hardware uses the input signal to select which register's data is placed on the register file's output bus, and state which of the two data output ports this data leaves through.



Output of last cycle now outputs and then inputs back to DIN1, so at this time, DIN1 <- value in Ra





Then there is another input AD3. AD3 determines to write DIN1, which is Ra's value, to address of Rb, and outputs Rb to DOUT3.

Extension questions

1. In a register-register MOV, what happens if $a = b$ (for example MOV R7, R7)? Does this cause any problems?

Add the instruction

MOV R7, R7

Nothing changes as DIN1 is R7's value (0 if unwritten), and AD3 is R7, so R7's value inputs back to R7.

	0	1	2	3	4	5	6	0
Datapath.DECODE.B(2:0)	7	0					0	7
Datapath.DECODE.A(2:0)	7	0					0	7
Reg16x8.REG7.Q(15:0)	x0000						x0000	x0000

As expected.